

Docker Fullstack esimerkki

Tässä esimerkissä rakennetaan sovellus, jossa on Express.js REST API + Postgres tietokanta + React sovellus, joita ajetaan Dockerissa. Sekä API, että React sovellus päivittyvät heti kun muutoksia tehdään.

Asennus

1. Nimeä .env.example tiedostot .env tiedostoiksi:

```
mv .env.example .env
mv frontend/.env.example frontend/.env
```

2. Käynnistä Docker Desktop
3. Rakenna imaget ja käynnistä kontit:
 - anna sovelluksen juurikansiossa komento

```
docker compose up --build
```

4. Avaa selaimeen sivu <http://localhost:3001/book> jolloin sinun pitäisi nähdä book-taulun data
5. Avaa selaimeen sivu <http://localhost:3000> jolloin sinun pitäisi nähdä React-sovelluksessa tietokannassa olevat kirjat
6. Kokeile muokata React sovelluksen App.js tiedostoa ja tutki päivittykö web-sivu

Buildaus

1. Kehityksessä sovellus buildataan komennolla

```
docker compose up --build
```

Jolloin suoritetaan sekä docker-compose.yml, että docker-compose.override.yml

2. Tuotannossa sovellus buildataan komennolla

```
docker compose -f docker-compose.yml up --build
```

Jolloin suoritetaan vain docker-compose.yml

Työskentely

- Kun haluat lopettaa kehityksen, anna komento **docker compose down** ja kun haluat jatkaa kehitystä anna komento **docker compose up --build**

- Jos teet muutoksia Dockerfile tai package.json tiedostoihin, sinun tulee käynnistää kontit uudelleen.

Tietokanta

Kaikki .sql ja .sh tiedostot, jotka on mountattu hakemistoon /docker-entrypoint-initdb.d/, ajetaan vain kerran konttia alustettaessa – eli silloin kun Postgres käynnistyy ensimmäistä kertaa ja data-hakemisto (/var/lib/postgresql/data) on tyhjä.

Koska **docker-compose.yml**-tiedostossa on rivi **./book.sql:/docker-entrypoint-initdb.d/book.sql:ro**, suoritetaan tiedostossa **book.sql** olevat koodit:

```
CREATE TABLE book (  
    id SERIAL PRIMARY KEY,  
    name VARCHAR(255),  
    author VARCHAR(255),  
    isbn VARCHAR(255)  
);  
  
insert into book (name, author, isbn) VALUES('Everything You Ever Wanted  
to Know','Upton','082305649x');  
insert into book (name, author, isbn)  
VALUES('Photography','Vilppu','205711499');  
insert into book (name, author, isbn) VALUES('Drawing Manual  
Vilppu','Zelanshi','1892053039');  
insert into book (name, author, isbn)  
VALUES('TBA','Zelanshi','0534613932');  
insert into book (name, author, isbn) VALUES('Shaping  
Space','Speight','0534613934');
```

Jos muokkaat tuota tiedostoa ja haluat, että se suoritetaan uudelleen, sinun on ajettava komento **docker compose down -v**

Muodostaaksesi yhteyden kontin Postgres-palvelimeen, suorita komento:

```
docker exec -it postgres_db psql -U netuser -d netdb
```

PostgreSQL

Tässä esimerkissä ei tarvita PostgreSQL serveriä, koska sitä ajetaan Dockerissa. Jos haluat kytkeytyä tietokantaan suoraan isäntäkoneelta tarvitset jonkin clientin, kuten **psql** tai pgAdmin. Jos haluat luoda dump-tiedostoja tarvitset **pg_dump** sovelluksen. Voit asentaa ne, kun lataat asennointi sovelluksen sivulta <https://www.postgresql.org/download/windows/> ja asennuksessa valitset asennettavaksi **Command Line Tools**, voit halutessasi asentaa myös graafisen clientin **pgAdmin 4**. Lisäksi kannattaa laittaa Windowsin ympäristömuuttujiin polku C:\Program Files\PostgreSQL\17\bin

Database dump

Voit luoda tietokannasta dumpin komennolla:

```
pg_dump -U netuser -h 127.0.0.1 -p 5432 netdb> dbdump.sql
```

Huomaa, että sinulla tulee olla asennettuna PostgreSQL, tai ainakin tuo pg_dump

Voit suorittaa edellä luodun dump-tiedoston komennolla:

```
psql -U netuser -h 127.0.0.1 -d netdb -f dbdump.sql
```

Dockerin kannalta oleelliset tiedostot

- docker-compose.yml
- docker-compose.override.yml(development tilassa)
- .env
- postgres.conf (jos halutaan saada paikalliseen PostgreSQL-palvelimeen tuotantopalvelinta vastaavat asetukset)
- api/Dockerfile
- frontend/Dockerfile
- frontend/.env

SSL-yhteys Node.js:n ja PostgreSQL:n välillä

Kun Node.js-sovellus yhdistää PostgreSQL-tietokantaan, yhteys voidaan suojata SSL/TLS-salauksella. Tämä tarkoittaa, että tieto kulkee salattuna palvelimen ja sovelluksen välillä.

Yleinen tapa hallita SSL-asetusta eri ympäristöissä on käyttää ympäristömuuttujaa, esimerkiksi:

DB_SSL=true tai **DB_SSL=false**.

Tällöin sama koodi toimii sekä kehityksessä että tuotannossa ilman muutoksia.

rejectUnauthorized - mitä se tarkoittaa?

Asetus **rejectUnauthorized** määrittää, tarkistaako Node.js palvelimen SSL-sertifikaatin luotettavuuden.

- **rejectUnauthorized: true** → Node tarkistaa, että palvelimen sertifikaatti on luotettava ja oikein allekirjoitettu
- **rejectUnauthorized: false** → Node ei tarkista sertifikaattia, vaikka yhteys olisi SSL-salattu

Kun arvo on true, yhteys on turvallisempi, koska Node varmistaa että se yhdistää oikeaan palvelimeen eikä väliin ole asettunut hyökkääjä.

Esimerkki turvallisesta asetuksesta

```
const pool = new Pool({
  connectionString: process.env.DATABASE_URL,
  ssl: {
    rejectUnauthorized: true
  }
});
```

Mitä PEM / CA-tiedosto tekee?

Joissain tapauksissa Node.js ei automaattisesti luota palvelimen sertifiikaattiin. Tällöin tarvitaan CA (Certificate Authority) -tiedosto, yleensä muodossa **ca.pem**.

Tämä tiedosto kertoo Node.js:lle, että tietty sertifiikaatti on luotettava.

PEM-tiedoston lisääminen koodiin

Jos palvelin käyttää omaa tai itse allekirjoitettua sertifiikaattia, CA-tiedosto lisätään näin:

```
const fs = require('fs');

const pool = new Pool({
  connectionString: process.env.DATABASE_URL,
  ssl: {
    ca: fs.readFileSync('/path/to/ca.pem').toString(),
    rejectUnauthorized: true
  }
});
```

Milloin PEM-tiedostoa EI tarvita?

PEM-tiedostoa ei yleensä tarvita, jos käytät pilvipalvelun (esimerkiksi Render, AWS tai Google Cloud) hallittua PostgreSQL-tietokantaa, koska ne käyttävät julkisesti luotettuja sertifiikaatteja.

Yhteenveto

- **rejectUnauthorized: true** → turvallinen ja suositeltu tuotannossa
- **rejectUnauthorized: false** → sallii yhteyden ilman sertifiikaatin tarkistusta (vain kehitykseen)
- **ca.pem** → tarvitaan vain, jos Node ei automaattisesti luota palvelimen sertifiikaattiin

Deploy Renderiin

Render ei osaa lukea docker-compose.yml -tiedostoa ja ajaa kaikkia palveluita yhdellä komennolla. Render tukee kuitenkin Dockerfilea, eli voit ajaa yhden kontin kerrallaan suoraan Dockerfilestä. Joten jaetaan sovellus osiin seuraavasti:

1. PostgreSQL-tietokanta

2. Backend
3. Frontend

Aluksi kannattaa luoda Renderiin PostgreSQL tietokanta ja luoda siihen samanlaiset taulut kuin omalla koneella (esim. dump-tiedoston avulla).

Deploy Docker Hubin kautta

Backend

1. Suorita komennot

```
docker build -t myusername/docker_example-api:latest api/  
docker push myusername/docker_example-api:latest
```

2. Renderissä:

- luo uusi WebService ja valitse Existing image ja kirjoita Image URL (=myusername/docker_example-api)
- Lisää Environment Variablesiin DATABASE URL ja valuen kopioit Renderin Postgressistä (kohdasta Connect)

Frontend

1. Suorita komennot

```
docker build -t myusername/docker_example-frontend:latest frontend/  
docker push myusername/docker_example-frontend:latest
```

2. Renderissä:

- luo uusi WebService ja valitse Existing image ja kirjoita Image URL (=myusername/docker_example-frontend)
- Lisää Environment Variablesiin REACT_APP_API_URL ja kopioi siihen backendisi URL (katso ettei loppuun tule kauttaviivaa)

Autentikointi

[autentikointi](#)

XSS hyökkäys

[xss-hyökkäys](#)